Лекция 5
Макросы

Clojure runtime · code as data · macroexpansion

Можно ли реализовать unless как функцию?

```
(defn unless-fn [condition body]
  (if (not condition) body))

(unless-fn true (println "Surprise!"))
```

Что напечатает REPL?

Почему функция не подходит?

```
(unless-fn true
  (println "Surprise!"))

;; Surprise!
;; => nil
```

Аргументы функции вычисляются до ее вызова.
Функция не управляет вычислением body.

01

Clojure runtime и REPL

Как текст программы преобразуется в исполняемый код?

REPL: Read–Eval–Print Loop

Read → Compile → Execute → Print

Что такое форма (form)?

```
42              ; число
x               ; символ
:ok             ; keyword
[1 2]           ; вектор
{:x 1}          ; map
(+ x 1)         ; список
```

Форма — синтаксическая единица Clojure.
Reader читает её как данные. Формы могут быть вложенными.

Reader: текст → данные

```
(read-string "(+ 1 2)")
;; => (+ 1 2)
(read-string "(fn [x] (* x x))")
;; => (fn [x] (* x x))
```

Reader построил списки. Функция + ещё не вызвана.
Анонимная функция не скомпилирована

Reader не компилирует

```
((read-string "(fn [x] (* x x))") 5)
;; => ClassCastException: PersistentList cannot
be cast to clojure.lang.IFn
```

Результат read-string еще не функция!

Compiler: данные → байткод

```
(eval (read-string "(fn [x] (* x x))"))
;; #object[user$eval136$fn__137 0x512d92b
user$eval136$fn__137@512d92b]
```

eval компилирует список в объект-функцию и вычисляет форму -> возвращает объект

Compiler: данные → байткод

```
(def f
  (eval (read-string "(fn [x] (* x x))")))
;; => #'user/f

(f 5)
;; => 25
```

Теперь f хранит функцию - можно применить

Путь выполнения формы

Reader: преобразует текст в данные Clojure

Compiler: преобразует данные Clojure в JVM байткод

JVM: запускает байткод

02

Имена и пространства имен

Как символ x превращается в значение?

Symbol — имя, а не значение

```
(read-string "x")
;; => x

(type (read-string "x"))
;; => clojure.lang.Symbol
```

Reader создает объект Symbol, но не ищет его значение.

eval: что означает x?

```
(eval (read-string "x"))
;; Ошибка компиляции:
;; Unable to resolve symbol: x
```

Символ прочитан, но компилятор не может разрешить имя x.

def: имя получает связывание

```
(def x 100)
;; => #'user/x

(eval (read-string "x"))
;; => 100
```

def создаёт (или переиспользует) Var user/x и связывает его со значением 100.

Три сущности: Symbol, Var, значение

```
(symbol "x")
;; => x                          ; Symbol

(resolve (symbol "x"))
;; => #'user/x                   ; Var

(deref (resolve (symbol "x")))
;; => 100                        ; значение
```

resolve ищет Var по символу в текущем контексте имён.

Namespace — пространство имен

```
(ns demo.alpha)
(def x 100)

(ns demo.beta)
(def x 5)

[demo.alpha/x demo.beta/x]
;; => [100 5]
```

Два разных Var: demo.alpha/x и demo.beta/x.

Текущий namespace и квалификация

```
(ns demo.beta)
(ns-name *ns*)   ;; => demo.beta, текущий namespace

x                ;; => 5, поиск в текущем namespace
demo.alpha/x     ;; => 100, явная квалификация

(resolve (symbol "x"))
;; => #'demo.beta/x
```

Неквалифицированный x зависит от *ns*; demo.alpha/x указывает явно.

Алиасы пространств имен

```
(ns demo.text
  (:require [clojure.string :as s]))

(s/upper-case "hello")
;; => "HELLO"

(clojure.string/upper-case "hello")
;; => "HELLO"
```

Alias s позволяет использовать короткую квалификацию имени

eval не видит локальное окружение

```
(ns user)
(def x 100)

(let [x 42]
  (eval (read-string "x")))
;; => 100
```

В let имя x локальное, а eval разрешает x через текущий namespace.

Разрешение имен при компиляции

x	→ сначала локальное связывание → затем Var из контекста namespace
demo.alpha/x	→ Var указанного namespace

Нет подходящего имени → ошибка компиляции

03

Code as Data

Как представлять код обычными структурами Clojure?

Quote: код как данные

```
(+ 1 2)              ;; => 3
'(+ 1 2)             ;; => (+ 1 2)
(first '(+ 1 2))     ;; => +
```

Syntax-quote: построение кода с квалификацией символов

```
`(+ 1 2)
;; => (clojure.core/+ 1 2)
```

Syntax-quote разрешает символы относительно namespace.

Unquote: вычисление и подстановка

```
(let [x 40]
  `(+ x 2))
;; => (clojure.core/+ user/x 2)
```

```
(let [x 40]
  `(+ ~x 2))
;; => (clojure.core/+ 40 2)
;; вместо символа x подставляется значение
```

~ вычисляет выражение при построении шаблона.

Unquote: более сложный пример

```
(let [x 4 y 5]
  `(+ (* x y) 2))
;; => (clojure.core/+ (clojure.core/* user/x
user/y) 2)
```

```
(let [x 4 y 5]
  `(+ ~(* x y) 2))
;; => (clojure.core/+ 20 2)
;; выражение (* x y) вычисляется и
подставляется
```

Unquote-splicing: ~@

```
(let [args [1 2 3]]
  `(+ args))
;; => (clojure.core/+ user/args)

(let [args [1 2 3]]
  `(+ ~@args))
;; => (clojure.core/+ 1 2 3)
```

~@ вставляет элементы последовательности в список

eval: данные снова становятся кодом

```
(eval '(+ 1 2))
;; => 3

(let [x 4
      y 5
      code `(+ ~(* x y) 2)
     ]
  (println code)
  (eval code))
;; => (clojure.core/+ 20 2)
;; => 22
```

04

Механизм макрорасширения

Как программист может преобразовать форму до ее исполнения?

Что такое макрос?

Макрос — функция преобразования кода.

Вход: невычисленные формы Clojure

Выход: новая форма Clojure

Компилятор компилирует результат макроса как обычный код.
Так можно создавать новые синтаксические конструкции.

Функция vs макрос

`defn`
- аргументы вычисляются заранее
- возвращает значение

`defmacro`
- получает исходные формы
- возвращает новую форму программы
- полученная форма вычисляется

Собственная конструкция unless

```
(defmacro unless [condition & body]
  `(if (not ~condition) ;; unquote
     (do ~@body)))      ;; unquote-splicing

(unless true (println "Surprise!"))
;; => nil, печати нет, никаких сюрпризов
```

Код body вычисляется только в выбранной ветви if.

macroexpand-1: один шаг

```
(macroexpand-1
 '(unless (zero? x)
    (println "nonzero")))

;; => (if (clojure.core/not (zero? x))
;;      (do (println "nonzero")))
```

Макрос может порождать другой макрос

```
(defmacro unless-via-when [test & body]
  `(when (not ~test)
     ~@body))

(unless-via-when false :ok)
```

Результат первого расширения — вызов when.
Но when тоже является макросом.

Один шаг vs повторное расширение

```
(macroexpand-1
  '(unless-via-when false :ok))
;; => (clojure.core/when
;;       (clojure.core/not false) :ok)

(macroexpand
  '(unless-via-when false :ok))
;; => (if (clojure.core/not false)
;;       (do :ok))
```

macroexpand повторяет расширение, пока внешняя форма не перестанет быть вызовом макроса.

Почему вложенный when не раскрыт?

```
(macroexpand '(when true :ok))
;; => (if true (do :ok))

(macroexpand '(do (when true :ok)))
;; => (do (when true :ok))
```

Внешний do — специальная форма, не макрос. macroexpand не обходит подформы; компилятор обработает when позже.

Как компилятор использует макрос

Reader	читает форму
Compiler	распознаёт вызов макроса
Macro	форма → другая форма
Compiler	анализирует и компилирует результат
JVM	исполняет байткод

Инструменты и границы

```
macroexpand-1   один шаг верхнего расширения
macroexpand     повторяет верхнее расширение
Compiler        обрабатывает вложенные формы

Внутри defmacro обычно НЕ вызывают eval.
```

05

Использование переменных

Как использовать переменные в макросах?

Переменные при макрорасширении

```
(defmacro print-result [expr]
  (let [message "Result:"]
    `(do (println ~message)
         ~expr)))

(macroexpand-1 '(print-result (+ 2 3)))
;; => (do (clojure.core/println "Result:")
;;        (+ 2 3))
```

message — локальная переменная самого макроса.
Ее значение вставляется в создаваемую форму.

Локальные имена в генерируемом коде

```
`(let [x 10] (+ x 1))
;; => (clojure.core/let [user/x 10] ...)
(eval `(let [x 10] (+ x 1)))
;; => Syntax error macroexpanding clojure.core/let

`(~'x)
;; => (x)
`(let [~'x 10] (+ ~'x 1))
;; => (clojure.core/let [x 10]
;;      (clojure.core/+ x 1))
```

'x создает символ x, а ~ вставляет его в шаблон.
Так мы избегаем квалификации локального имени.

Локальные имена в генерируемом коде

```
`(let [~'x 10] (+ ~'x 1))
;; => (clojure.core/let [x 10]
;;      (clojure.core/+ x 1))
;; неквалифицированный символ x

(eval `(let [~'x 10] (+ ~'x 1)))
;; => 11
```

Макрос с локальной переменной

```
(defmacro calculate []
  `(let [~'x 10]
     (* ~'x 2)))

(macroexpand-1 '(calculate))
;; => (clojure.core/let [x 10]
;;      (clojure.core/* x 2))

(calculate)  ;; => 20
```

Макрос создает let, но сам его не выполняет.
Локальная переменная x появляется в созданном коде.

Переменные макроса и генерируемого кода

	Переменная макроса	Переменная генерируемого кода
Где определяется	`let` в теле `defmacro`	`let` внутри `syntax-quote`
Когда существует	При макрорасширении	При выполнении результата
Для чего нужна	Построение формы	Вычисления в созданной программе

06

Переменные и гигиена

Как сгенерированный код может изменить смысл исходной программы?

Захват переменной: проблема

```
(defmacro add-ten [expr]
  `(let [~'x 10]
     (+ ~'x ~expr)))

(macroexpand-1 '(add-ten x))
;; => (clojure.core/let [x 10] (clojure.core/+ x x))

(let [x 100] (add-ten x))
;; => 20, а не 110
```

Вставленное expr = x попало под локальное связывание x.

Захват переменной: автогенерация имен

```
(defmacro add-ten [expr]
  `(let [x# 10]
     (+ x# ~expr)))

(let [x 100] (macroexpand-1 '(add-ten x)))
;; => (clojure.core/let
;;      [x__8010__auto__ 10]
;;        (clojure.core/+ x__8010__auto__ x))

(let [x 100] (add-ten x)) ;; => 110
```

x# создаёт уникальный символ; в одном шаблоне повторения x# согласованы.

Multiple evaluation: ещё одна ошибка

```
(defmacro twice [expr]
  `(+ ~expr ~expr))

(twice (do (println "calc") 21))
;; calc
;; calc
;; => 42
```

Одно выражение в вызове → два исполнения сгенерированного кода.

Single evaluation: исправление

```
(defmacro twice [expr]
  `(let [value# ~expr]    ;; используем let
     (+ value# value#)))

(twice (do (println "calc") 21))
;; calc
;; => 42
```

let гарантирует однократное вычисление expr;
value# исключает capture.

Приёмы гигиены макросов

x# / gensym → генерировать уникальные символы

Syntax-quote → квалифицировать глобальные имена

let → вычислять выражение один раз

Гигиена в Clojure - ответственность программиста

07

Расширение языка

Строим свои абстракции

Хотим писать так

```
(with-retry 3
  (println "request")
  (fetch-data))
```

Не более трех попыток. При успехе вернуть значение, после 3х неудачных попыток - бросить исключение.

Можно ли обойтись функцией?

```
(retry-fn 3 (remote-call args))
   → remote-call вычисляется до retry-fn

(retry-fn 3 #(remote-call args))
   → работает: функция передана как thunk
```

Макрос нужен для удобной формы записи, не для retry.

Функциональное ядро: функция retry*

```
(defn retry* [max-attempts f]
  {:pre [(pos? max-attempts)]} ;; предусловие
  (loop [i 1]                  ;; цикл попыток
    (let [[status v]      ;; вектор статус + результат
          (try [:ok (f)]  ;; успех
               (catch Exception e [:error e]))];;неудача
      (cond
        (= status :ok) v                      ;; успех
        (< i max-attempts) (recur (inc i)) ;; повтор
        :else (throw v)))))   ;; не осталось попыток
```

Важно: recur расположен ВНЕ try; Exception ловится только из (f)

Тонкий макрос-обёртка

```
(defmacro with-retry [max-attempts & body]
  `(retry* ~max-attempts
           (fn [] ~@body)))

(with-retry 3 (remote-call args))
```

Тело не вычисляется до вызова retry*:
макрос оборачивает его в fn

Что видит компилятор после expansion?

```
(macroexpand-1
 '(with-retry 3 (remote-call args)))

;; => (user/retry* 3
;;      (fn [] (remote-call args)))
```

Имя user/retry* зависит от namespace определения макроса

Демо: успех на третьей попытке

```
(let [n (atom 0)]
  (with-retry 3
    (let [i (swap! n inc)]
      (println "attempt" i)
      (if (< i 3)
        (throw (ex-info "temporary" {}))
        :ok))))
;; attempt 1
;; attempt 2
;; attempt 3
;; => :ok
```

Весь body вызывается заново на каждой попытке.

Retry в распределенных системах

Idempotency → риск дублирования эффектов
Timeout → не ждать бесконечно
Backoff + jitter → не перегружать сервис
Retry predicate → отличать временные сбои от постоянных

Макрос определяет синтаксис, но не решает эти задачи.

08

Пример: Cypher-like DSL

Пишем свой DSL

Пример: топология сервисов

```
(def topology
  {:edges [[:api  :calls :auth]
           [:api  :calls :billing]
           [:auth :calls :db]]})

;; :api, :auth, :billing, :db идентификаторы
сервисов
;; Какие сервисы напрямую вызывает :api?
```

Граф — обычные данные Clojure: каждый вектор описывает ребро.

Cypher-like DSL: как хотим писать запрос?

```
(match topology
  (source :calls target)
  (where (= source :api))
  (return target))

;; => (:auth :billing)
```

source и target — переменные шаблона; **where** и **return** — маркеры синтаксиса DSL.

Как реализовать запрос?

```
(let [edges (:edges topology)]
  (for [[source relation target] edges
        :when (and (= relation :calls)
                   (= source :api))]
    target))

;; => (:auth :billing)
```

Наш match будет порождать обычный for

Реализация макроса match

```
(defmacro match [graph [from rel to]
                 [_ pred] [_ proj]]
  `(let [edges# (:edges ~graph)]
     (for [[~from relation# ~to] edges#
           :when (and (= relation# ~rel)
                      ~pred)]
       ~proj)))
```

Деструктуризация извлекает части запроса; relation# — безопасное имя.

macroexpand-1: результат — код Clojure

```
(macroexpand-1
 '(match topology
    (source :calls target)
    (where (= source :api))
    (return target)))
;;  (clojure.core/let
;;    [edges__7897__auto__ (:edges topology)]
;;    (clojure.core/for
;;      [[source relation__7898__auto__ target] edges__7897__auto__
;;        :when (clojure.core/and
;;          (clojure.core/= relation__7898__auto__ :calls)
;;          (= source :api))]
;;      target))
```

Макрос порождает let с вложенным for

Границы учебного DSL

```
;; НЕ поддерживается несколько рёбер:
(match topology
  (a :calls b)
  (b :calls c)
  (where (= a :api))
  (return c))
```

**Для такого запроса нужны разбор нескольких шаблонов,
генерация связываний
и проверка синтаксиса where / return.**

Итог: код как данные → расширение языка

`forms` → `macroexpand` → `compile` → `execute`

Макросы расширяют синтаксис; функции реализуют вычислительную семантику.

Когда нужен макрос?

Можно передать значение? → `defn`

Можно передать функцию? → `defn`

Нужен собственный синтаксис, управление вычислением форм? → `defmacro`

Рассматриваем сначала функцию. Макрос — когда оправдан.